

SecureBERT and LLAMA 2 Empowered Control Area Network Intrusion Detection and Classification

Xuemei Li^{ID}, *Student Member, IEEE*, and Huirong Fu^{ID}, *Senior Member, IEEE*

Abstract—Numerous studies have proved their effective strength in detecting Control Area Network (CAN) attacks. In the realm of understanding the human semantic space, transformer-based models have demonstrated remarkable effectiveness. Leveraging pre-trained transformers has become a common strategy in various language-related tasks, enabling these models to grasp human semantics more comprehensively. To delve into the adaptability evaluation on pre-trained models for CAN intrusion detection, we have developed three distinct models: CAN-C-BERT, CAN-SecureBERT and CAN-LLAMA2. Notably, our CAN-LLAMA2 model surpasses the state-of-the-art models by achieving an exceptional performance 0.999993 in terms of Balanced Accuracy (BA), Precision (PREC), Detection Rate (DR), F1 score, and a remarkably low false alarm rate of 3.10×10^{-6} . Impressively, the false alarm rate is 52 times smaller than that of the leading model, MTH-IDS (Multitiered Hybrid Intrusion Detection System). Our study underscores the promise of employing a Large Language Model as the foundational model, while incorporating adapters for other cybersecurity-related tasks and maintaining the model's inherent language-related capabilities.

Index Terms—Transformer, CAN-C-BERT, CAN-SecureBERT, CAN-LLAMA2, vehicle cybersecurity.

I. INTRODUCTION

VEHICLE CAN constitutes a standardized communication protocol extensively employed within the automotive industry. It serves as a primary communication interface connecting the vehicle gateway, Electronic Control Units (ECUs), and various other control components integral to vehicular operations. For an Intrusion Detection System (IDS), monitoring the CAN messages between ECUs to enable a comprehensive understanding of the communication processes and the identification of anomalous messages are essential. The United Nations Regulation WP.29 R155 [1] has articulated a new mandate, rendering it obligatory for all new vehicles manufactured within the European Union to adhere to this regulation, effective from July 2024. This regulation enforces all vehicles to possess the capability to detect and respond to potential cybersecurity attacks. Moreover, there is a requisite for the systematic collection of log data to facilitate the identification of cyber attacks and to support forensic investigations. Therefore, an adaptable and multifaceted solution capable of meeting both demands becomes imperative.

Received 14 November 2023; revised 22 June 2024, 17 December 2024, 20 April 2025, and 18 June 2025; accepted 21 July 2025. Date of publication 13 August 2025; date of current version 21 October 2025. This work was supported in part by U.S. National Science Foundation under Grant 2146280, Grant 2327944, and Grant 2349663. The Associate Editor for this article was H. H. Song. (Corresponding author: Xuemei Li.)

The authors are with the Department of Computer Science and Engineering, Oakland University, Rochester, MI 48309 USA (e-mail: xuemeili@oakland.edu).

Digital Object Identifier 10.1109/TITS.2025.3596915

Based on technology implementation, four primary methodologies for CAN intrusion detection have been identified, visualized in Figure 1 [2]. These methodologies are proficient in detecting CAN intrusion attacks but do not possess the inherent capability to collect and analyze log data. The first is fingerprint-based, which involves the detection of anomalies through clock-based or voltage measurements. The second, performed at the message level, is parameter monitoring-based and comprises techniques such as frequency-based, whitelist-based, or remote frame-based analysis. The third operates at the data-flow level and is grounded in information theory, employing entropy analysis or hamming distance measurements. The fourth takes advantage of Machine Learning (ML) techniques at a functional level. ML methodologies have undergone extensive evaluation in the context of CAN intrusion detection, with a focus on Recurrent Neural Networks (RNNs), Deep Neural Networks (DNNs), and Artificial Neural Networks (ANNs). Additionally, ML models like Decision Tree (DT)-based or Hidden Markov Model (HMM)-based models have found application in this domain.

The primary limitation of existing implementations in CAN intrusion detection is their dependence on supplementary access, such as physical layer interactions, preprocessing of CAN messages, extraction of corresponding features, and creation of intricate rules and work flows within software, which leads to numerous additional requirements for effective CAN intrusion detection, while limiting the capacity to identify unknown and novel attacks. To address these constraints, Cho and Shin [3] developed Viden, a novel approach that entails fingerprinting ECUs based on their corresponding voltage measurements. Viden acquires precise voltage measurements directly from the message transmitters and subsequently processes this data to construct accurate voltage profiles and fingerprints. The method is primarily applicable to physical layer attacks. Furthermore, frequency-based techniques adopt a different perspective by focusing on periodic traffic, rooted in the observation that frequencies tend to increase when malevolent adversaries engage in spoofing or Denial of Service (DoS) attacks [4]. On the other side, CAN IDS using the traditional ML models such as Decision Trees (DTs), Random Forests (RFs), and Neural Networks, require preprocess CAN message and perform feature engineering to train the model in order to perform classification.

Transformer models have garnered recognition for the remarkable proficiency in grasping human semantics and handling various Natural Language Processing (NLP) tasks. They possess the capacity to accept textual input and generate output directly. Their remarkable capabilities have been well-

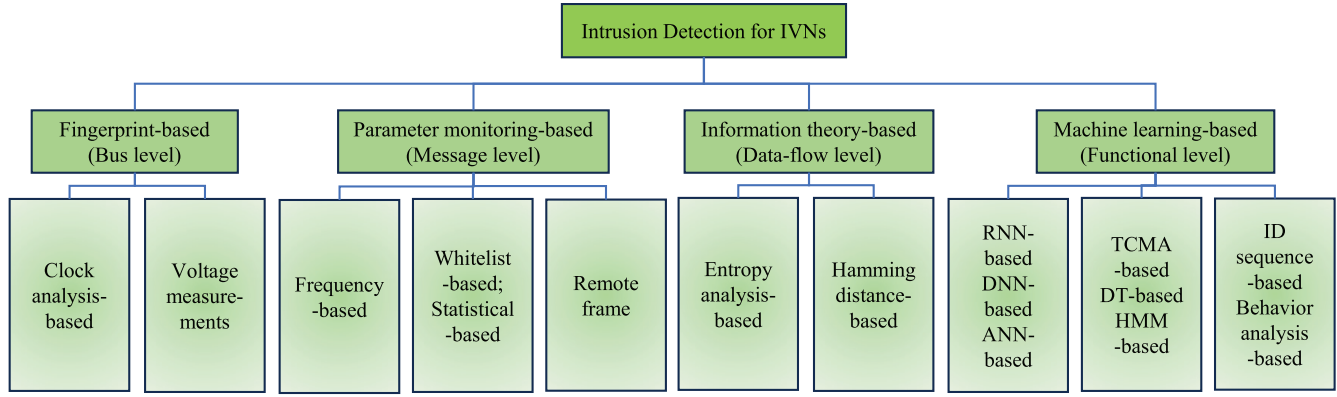


Fig. 1. Taxonomy of Intrusion Detection Methods in In-Vehicle Networks (IVNs), categorized by underlying technology implementation: fingerprint-based, parameter monitoring-based, data-flow-based, and machine learning-based approaches. This classification highlights the diversity in detection strategies, each targeting different layers of the CAN communication stack [2].

demonstrated in the realms of computer vision and NLP tasks. The first introduce of the transformer model can be attributed to Vaswani et al. [5]. Its architecture comprises an encoder and decoder structure. This design replaced the traditional recurrent-based models' recurrent layer with a multi-head self-attention layer, leading to substantial improvements in performance, particularly in translation tasks.

There are a couple of motivations for our study to integrate transformer models. First, transformer models can be pre-trained on extensive unlabeled datasets. This pre-training can significantly enhance model performance during transfer learning [6]. This technique is commonly referred to as domain adaptation, which falls under transductive transfer learning. It is specialized to address challenges arising from a substantial volume of unlabeled source data for training and limited unlabeled target data [7]. This strategy has demonstrated its efficacy by achieving state-of-the-art results in various common NLP benchmarks, as evidenced in literature [8]. The empirical robustness of this strategy extends across diverse domains, including computer vision, audio, and text processing tasks.

This paper endeavors to adapt pre-trained transformer-based models for the purpose of detecting CAN attacks. Leveraging the inherent capabilities of transformers has the potential to markedly enhance efficiency and streamline the architecture of an IDS for vehicular CAN attacks. Transformers offer a range of advantages, the first being their ability to learn intrinsic relationships through position encoding and a multi-head self-attention mechanism. The second advantage is that transformers can learn meaningful representations from CAN message logs without the need for extensive data preprocessing or feature extraction.

In this study, **Our Primary Contributions** are summarized as follows:

- 1) We propose CAN-C-BERT, CAN-SecureBERT and CAN-LLAMA2 models to detect and classify various CAN attacks by adapting pre-trained transformer-based models, namely BERT [9], SecureBERT [10], and LLAMA 2 [11].

- 2) We design and demonstrate detailed architectures of the proposed three models and discuss how to train them to detect and classify CAN attacks.
- 3) We develop and construct the proposed CAN-C-BERT, CAN-SecureBERT and CAN-LLAMA2 models by integrating pre-trained models with classification heads and training the proposed models with pre-balanced CAN dataset.
- 4) We conduct empirical study of the proposed models and compare them with the state-of-the-art model MTH-IDS [12]. The experimental study shows that our proposed CAN-LLAMA2 model demonstrates the highest performance and CAN-SecureBERT secures the second-highest position.

In particular, our work has unique innovations which yield significant results, including:

- 1) **Utilization of CAN Message Logs:** Our proposed CAN-C-BERT, CAN-SecureBERT and CAN-LLAMA2 models possess the unique ability to directly employ CAN message logs for intrusion detection and attack classification. This approach eliminates the need for traditional data preprocessing, as the models can directly analyze the raw data.
- 2) **Superior Performance with Limited Data and Enhanced Generalization with Larger Data:** Our proposed CAN-C-BERT, CAN-SecureBERT and CAN-LLAMA2 models have achieved superior performance while training only 10% of the data, while other state-of-art models ([13], [14], [15], [16], [17], [18], [19], [20], [21]) require a larger dataset to achieve a performance level similar to ours. We found more training data used will contribute to model performance improvement but the improvement magnitude gradually reduce as training data increases. Furthermore, our models trained with more extensive datasets exhibit improved generalization and outperform their counterparts trained with smaller dataset.
- 3) **Better Model Performance with More Complex Model Architecture:** By comparing the performance

of CAN-C-BERT and CAN-SecureBERT, we do not observe a significant performance difference. This suggests that a transformer pre-trained with cybersecurity domain knowledge does not necessarily improve the capability to detect attacks. Model ability to capture attack patterns depends on the complexity of the model architecture.

- 4) **LoRA Utilization in CAN-LLAMA2:** Leveraging the Low-Rank Adaption (LoRA) [22] technique for training CAN-LLAMA2 resulted in a modification of only 0.57% of the model's parameters. This demonstrates that the majority of the original LLAMA 2 model parameters remain unchanged. Consequently, the CAN-LLAMA2 model retains its versatility and can be employed for various language-related tasks. For instance, the Vehicle Security Operations Center (VSOC) team can capitalize on this model and adapt it for diverse downstream tasks by fine-tuning the pre-trained model and integrating adapter heads.

The remainder of this paper is organized as follows: Section II presents an extensive literature review. Section III conducts a comparative analysis of the model architectures, highlighting the distinctions between CAN-C-BERT, CAN SecureBERT, and CAN-LLAMA2. Section IV outlines the key techniques employed for fine-tuning pre-trained models. Section V furnishes in-depth information regarding the datasets and the training equipment employed in our research. Section VI showcases our experimental results, addresses our research questions, and offers a performance comparison of our proposed models with state-of-the-art models. Finally, Sections VII and VIII conclude and delineate future work.

II. RELATED WORKS

In this section, we identify the most current research endeavors that employ transformer-based models for CAN attack detection. We aim to provide a comprehensive overview of the latest findings and also assess the inherent limitations of these relevant studies.

Nwafor et al. proposed a language-based intrusion detection model utilizing BERT [23]. They first trained the BERT model to comprehend the semantics within CAN messages. Subsequently, they fine-tuned the model for CAN message classification. The training procedure entailed 64% of the data, while 20% of the data was allocated for validation, and the remaining 16% was dedicated to testing. Their model achieved close to 100% accuracy, precision, Recall, and F1 score. However, the specific details of their performance were not reported.

In a related study, Natasha et al. introduced the "CAN-BERT" model [24]. Their primary objective was to design an anomaly detection model and frame the CAN message classification as a binary classification problem, differentiating between normal and abnormal messages. They adopted the BERT model and trained it using standard CAN messages, incorporating Masked Language Model (MLM) techniques. Subsequently, the model predicted the probability distribution for each CAN ID in randomly masked testing sequences. Their approach classified messages as abnormal if they lacked

any CAN ID associated with a normal message. Their model achieved F1-scores ranging from 0.81 to 0.99 for different types of attacks. However, their approach heavily relied on the probability distribution of normal message CAN IDs, rendering it incapable of detecting injected message attacks that employ similar CAN IDs as normal messages. Furthermore, their model exhibited an F1 score below 0.9 for fuzzy and malfunction attacks. Another limitation of their approach is its binary classification nature, necessitating the development of an additional anomaly classification model.

Aghaei et al. introduced SecureBERT in their study [10], presenting a cybersecurity language model designed to capture text connotations within cybersecurity-related texts, such as Cyber Threat Intelligence (CTI). SecureBERT is constructed upon a pre-trained Roberta model, featuring a custom tokenizer that incorporates the original Roberta model token dictionary with cybersecurity domain tokens. This model was trained on an extensive corpus of cybersecurity text and was assessed using standard MLM methods. SecureBERT serves as a valuable pre-trained model imbued with domain-specific knowledge in the cybersecurity field, which we utilize in our research to develop CAN-SecureBERT.

In a separate effort, Touvron et al. from Meta GENAI unveiled LLAMA 2 [11], a novel family of pre-trained and fine-tuned models with scales ranging from 7 billion to 70 billion parameters. The models were constructed based on the conventional transformer model as introduced in [5]. LLAMA 2 models incorporate pre-normalization using RMSNorm, employ the SwiGLU activation function, implement rotary positional embeddings, and feature grouped query attention. These models were trained using preprocessed data amounting to 2000 billion tokens, thereby encapsulating a substantial wealth of knowledge in comparison to SecureBERT. In our study, we will use LLAMA 2 to develop CAN-LLAMA2.

Khandelwal and Shreejith [25] proposed an FPGA-based IDS framework for Electronic Control Units (ECUs), achieving 99% detection accuracy while significantly reducing energy consumption and inference latency. Their work provides a strong benchmark for resource-efficient IDS deployment, particularly in embedded automotive environments.

Similarly, Huan et al. [26] introduced a lightweight deep learning model utilizing a T-shaped temporal windowing strategy, enabling a favorable trade-off between detection performance and computational overhead. Their architecture is well-suited for real-time deployment in low-resource environments.

Together, these works highlight the growing diversity of scalable IDS designs, spanning from high-performance LLM-based models to lightweight, hardware-friendly solutions. Such perspectives are critical in shaping adaptable IDS frameworks for heterogeneous automotive platforms.

Consequently, as indicated by the above literature review, there is an absence of prior research endeavors that have previously employed SecureBERT and LLAMA 2 for the purpose of CAN intrusion detection and classification. Therefore, our research aims to fill a significant gap in this domain.

TABLE I
PRE-TRAINED MODEL COMPARISON

Model Type	Model Size (GB)	Pre-training Data	Architecture	Key Features
BERT	0.42	BooksCorpus, English Wikipedia	Transformer Encoder	Bidirectional, Next Sentence Prediction
SecureBERT	0.48	Common Crawl, Books, Wikipedia, 98,411 Cybersecurity Related Documentation	Transformer Encoder	Robust training on larger data
LLAMA2	12.5	2 Trillion Tokens of Publicly Available Data	Transformer Decoder	Few-shot learning capabilities, large model size

III. MODEL ARCHITECTURE

In this section, we present a comprehensive overview of transformer architecture and the model architectures of our proposed CAN-C-BERT, CAN-SecureBERT and CAN-LLAMA2 models. We build our proposed model on top of pre-trained transformer models BERT, SecureBERT and LLAMA 2. The interpretability of how each of the proposed models generates the decision-making process for CAN attack detection and classification will be described and discussed. The overview to compare them is listed in Table I.

A. Transformer

The Transformer deep learning architecture, first proposed by Google in 2017, has served as the base model for numerous NLP LLMs, including BERT, GPT, and LLAMA 2. Figure 2 illustrates the model architecture, as presented in [5], comprising an encoding and a decoding phases. The transformation process comprises 6-stack encoders and decoders, respectively. It is akin to “Transformers” components disassemble and subsequent reassemble process. Each stack process was subdivided into eight blocks based on different initial values. Each block possesses a “transformation manual”, recording component weights and mutual relationships.

Each encoder consists of two core components: a multi-head self-attention mechanism and a feed-forward neural network. Input embeddings are generated via tokenization, with positional encodings added to retain token order. These embeddings pass through the multi-head self-attention layer, which performs eight parallel computations using the Key (K), Query (Q), and Value (V) vectors to capture relationships between tokens. The attention output is then processed by a feed-forward layer. Both layers are enhanced with residual connections and layer normalization for stability and improved learning.

The decoder mirrors the encoder’s structure but introduces key differences. It includes masked multi-head attention to prevent tokens from attending to future positions—an essential feature for autoregressive tasks. Additionally, it applies attention over the encoder’s output using Ks and Vs from the encoder and Qs from the decoder. The result is passed through a softmax function, which selects the most probable next token in the sequence.

The training methodology of the Transformer network adheres to the gradient descent algorithm, leveraging back-propagation to adjust model parameter weights by minimizing

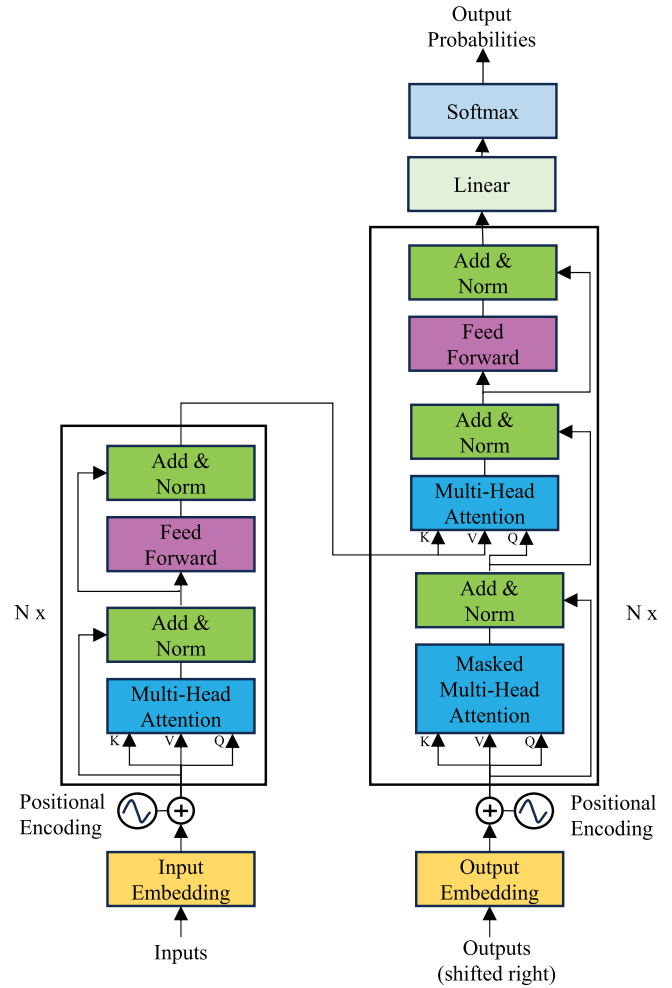


Fig. 2. Overview of the Transformer model architecture, as introduced in [5], featuring stacked encoder and decoder layers with self-attention and feed-forward modules. The figure highlights key components such as multi-head attention, positional encoding, and masked decoding, which together enable effective sequence modeling in natural language processing tasks. [5].

the error between predictions and actual values, thus achieving optimal learning outcomes.

B. CAN-C-BERT

BERT, which stands for Bidirectional Encoder Representations from Transformers, is an “Encoder-only” transformer introduced by Google in 2018. It undergoes training on large-scale datasets, and its robust pre-training enables fine-tuning

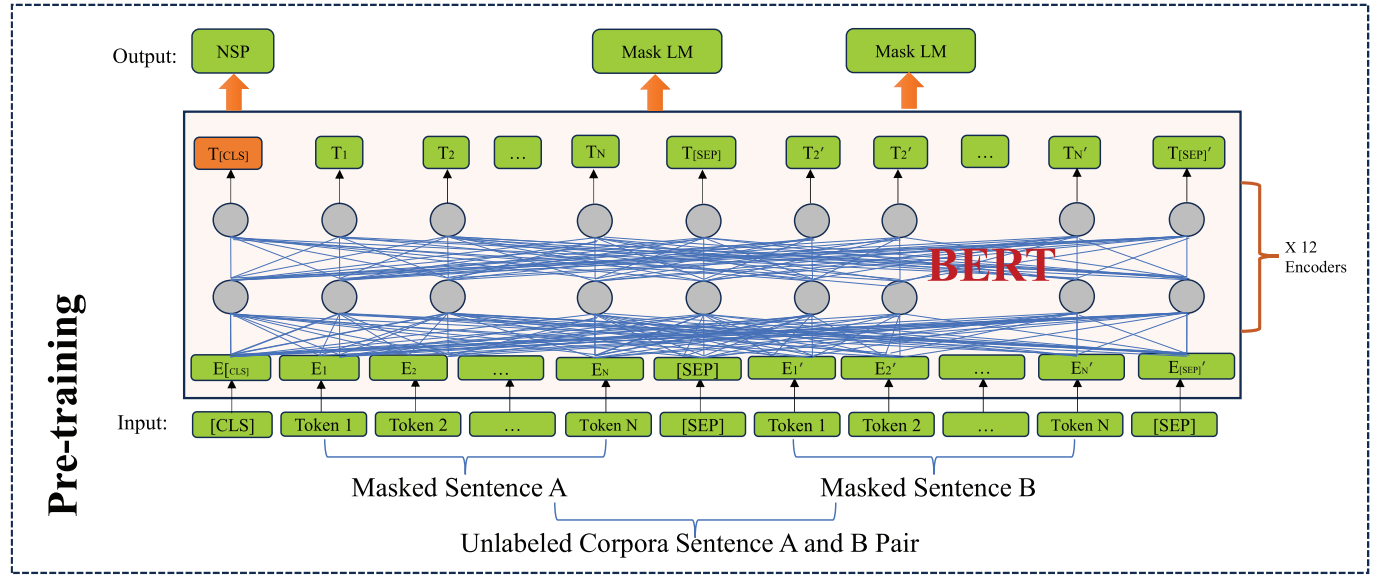


Fig. 3. Illustration of the BERT pre-training process, which includes Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). These tasks enable the model to learn deep bidirectional representations of text by predicting masked tokens and understanding sentence relationships using large-scale unlabeled corpora [9].

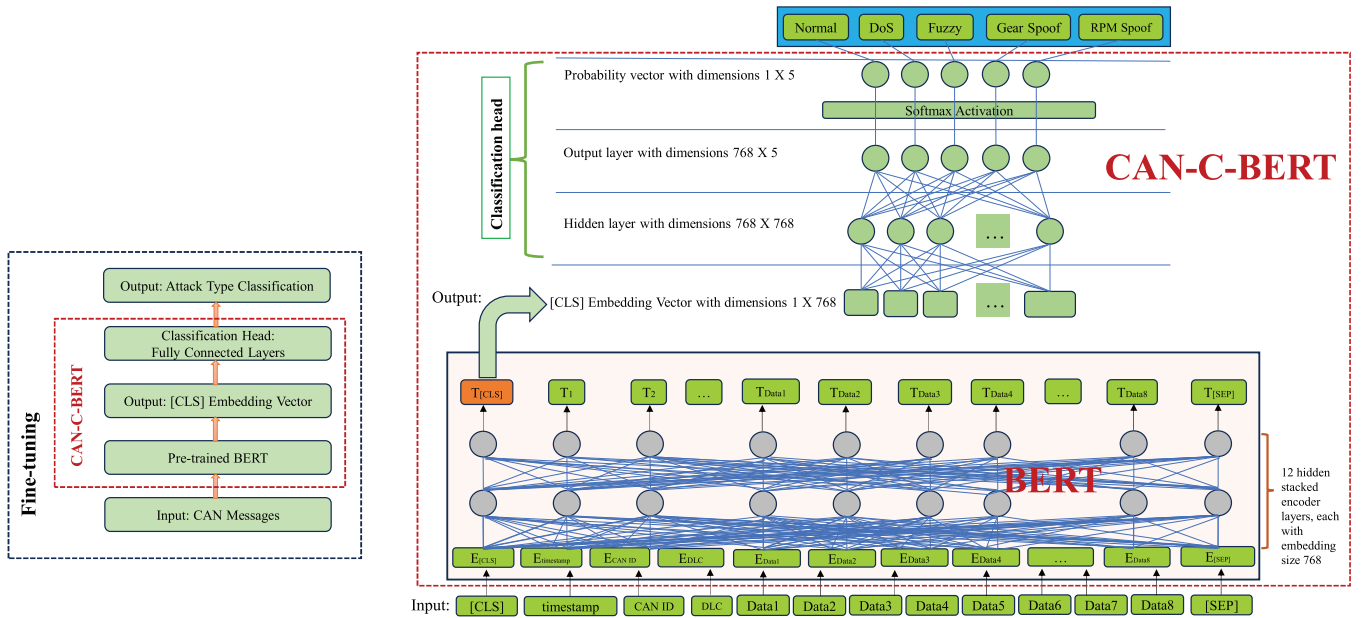


Fig. 4. Architecture of the CAN-C-BERT model, consisting of a pre-trained BERT base encoder with 12 transformer layers and a classification head. The red highlighted region shows the fine-tuning phase used for CAN intrusion detection, adapting BERT's language representations to the classification of CAN bus messages.

for various downstream tasks, highlighting its versatility and effectiveness in natural language understanding.

BERT models are designed to generate deep bidirectional representations from unlabeled text by considering both left and right context across all layers. After pre-training, a BERT model can be fine-tuned for various downstream tasks, such as sequence classification, by adding a single additional output layer. In this study, we employ BERT base version model, which has been pre-trained on the BookCorpus dataset [27]. This dataset comprises 11,038 unpublished books and

English Wikipedia. The BERT pre-training process is shown in Figure 3.

The model architecture of CAN-C-BERT model are depicted in Figures 4. We name this fine-tuned intrusion classification model “CAN-C-BERT”. The “C” in the middle of “CAN” and “BERT” is short for “Classification”. The architecture of the CAN-C-BERT model is particularly emphasized within the red Fine-tuning box. It incorporates a pre-trained BERT base version model and a classification head. The pre-trained BERT base version model has 12 layers of transformer

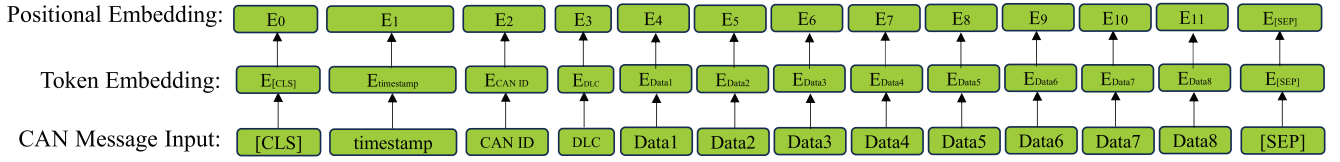


Fig. 5. CAN-C-BERT Embedding Input Structure: The BERT model combines Tokenization Embedding (1, n, 768) and Position Embedding (1, n, 768) through summation to obtain an input Embedding (1, n, 768) for the model.

encoder blocks totaling 110 million model parameters. The classification head is implemented as a fully connected neural network.

The first step to obtain CAN-C-BERT model is the pre-training process. The pre-training process establishes the foundational understanding of grammar and context, known as language representation. This process is characterized by its key feature—the utilization of unsupervised corpora, including consecutive sentences. The CAN-C-BERT embedding input structure is shown in Figure 5. The positional encoding information is added after segment embedding and tokenization embedding. BERT employs two primary training strategies: MLM and Next Sentence Prediction (NSP). In MLM, each sentence masks some parts of its words, and predicting the masked words based on the context of bidirectionality, inspired by human cloze tests. After transformation, the embedding output is obtained, and the output “T N” represents the prediction of the masked word. In NSP, the original input is modified to include a special token ([CLS]) at the beginning, with sentence separation indicated by the [SEP] token.

The second step involved is the fine-tuning process. The fine-tuning process involves additional training tailored to specific tasks. A BERT base model consists of 12 layers of transformer encoder blocks, 768 hidden layers, and 12 self-attention heads. To train a BERT-based model for sequence classification tasks, a classification head is integrated into the embedding vector of the classification task token [CLS] derived from the BERT model. The [CLS] token summarizes the sentence and represents the output of the last layer for subsequent classification. The classification head shown in Figure 4 comprises fully connected neural networks, featuring a hidden layer, an output layer, and a softmax activation layer. The [CLS] embedding vector passes through the hidden layer and connects to the subsequent output layer. The output from the output layer is then processed through a softmax activation layer to generate the probability vector. The class with the highest probability is subsequently considered the final predicted output.

For CAN attack detection and classification, the process initiates with the tokenization of the CAN messages. These tokens are then input into the pre-trained BERT model, and the output embedding for the [CLS] token becomes the input for the classification head. During training, the cross-entropy loss is computed by comparing the predicted labels to the actual ground truth labels.

C. CAN-SecureBERT

The process using CAN-SecureBERT to classify CAN messages mirrors the approach employed by CAN-C-BERT. It

consists of a pre-trained SecureBERT model and a classification head, which is embodied by a fully connected neural network. SecureBERT leverages the architecture of a pre-trained RoBERTa-base model, featuring 12 hidden transformer and attention layers, in addition to one input layer. This adaptation involves fine-tuning the RoBERTa-base model utilizing a substantial dataset of 98,411 cybersecurity-related textual elements (equivalent to 1 billion tokens). The model integrates a customized tokenizer based on the original RoBERTa tokenizer, effectively expanding the overall vocabulary to 50,265. This tailored tokenizer enhances the model’s proficiency in extracting cybersecurity-related tokens from textual corpora. Like its name, SecureBERT compounds Security and BERT. The model’s efficacy is further augmented by introducing noise into the token weights of the vocabulary during the training phase.

The model architecture of CAN-SecureBERT is the same as CAN-C-BERT. It incorporates a pre-trained SecureBERT model and a classification head. The pre-trained SecureBERT model has 12 transformer blocks totaling 123 million model parameters. The classification head is implemented as a fully connected neural network. It is incorporated after the embedding vector of the classification token [CLS] is derived from the SecureBERT model. It comprises a fully connected neural networks including a hidden layer, an output layer, and a softmax activation layer. The [CLS] embedding vector passes through the hidden layer and then connects to the output layer. The output from the output layer is passed into a softmax activation layer to obtain the probability vector. The class with the highest probability is the final predicted output.

D. CAN-LLAMA2

LLAMA2, the second iteration of the LLAMA series, represents a collection of generative text models that are pre-trained and fine-tuned. It was officially introduced on July 18, 2023, as a collaborative initiative between Meta and Microsoft. The LLAMA 2 models, developed and launched by Meta, are available in three distinct sizes with parameter counts of 7 billion, 13 billion, and 70 billion. These models were trained on an extensive dataset comprising 2 trillion tokens sourced from various outlets such as web pages (CommonCrawl), open-source repository code (GitHub), Wikipedia content (in 20 different languages), public domain books, Latex source code (from scientific papers on ArXiv), and questions and answers from Stack Exchange. The dataset curation involved meticulous removal of websites containing personal data and the up-sampling of samples from reliable sources, as detailed in [11].

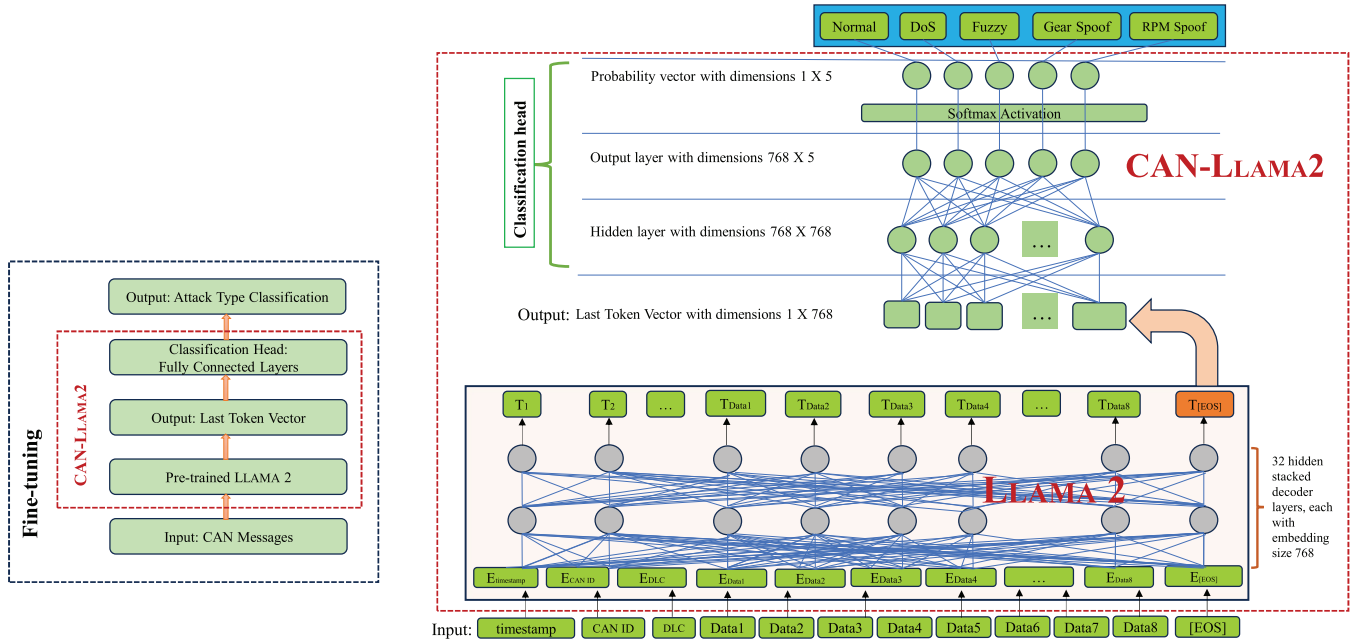


Fig. 6. Architecture of the CAN-LLAMA 2 fine-tuning model, which integrates a pre-trained LLAMA 2 decoder-based language model (7B parameters, 32 transformer blocks) with a classification head for CAN attack detection. The classification head processes the [EOS] token embedding to predict attack types via a fully connected network followed by softmax activation. This framework adapts LLAMA 2's generative capabilities for token-level classification in automotive intrusion detection tasks.

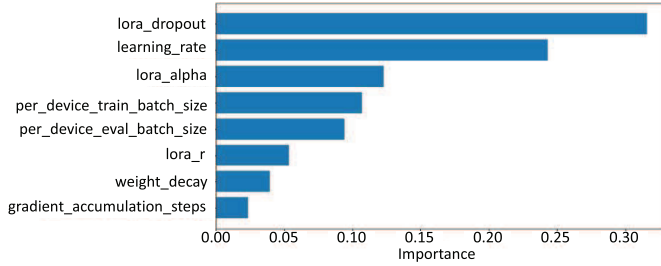


Fig. 7. Hyperparameter importance of CAN-LLAMA2.

The model architecture of the CAN-LLAMA2 model is depicted in Figure 7. CAN-LLAMA2 model consists of a pre-trained LLAMA 2 model and the classification head. The pre-trained LLAMA 2 model utilized in this study is a half-precision model. It has 32 transformer decoder blocks totaling 7 billion model parameters. The classification head is implemented as a fully connected neural network.

For training the pre-trained LLAMA 2 model for CAN attack detection and classification tasks, a classification head is integrated into the embedding vector of the last token ([EOS]) (End of the Sentence) derived from the LLAMA 2 model. This classification head includes a fully connected neural network with a hidden layer, an output layer, and a softmax activation layer. The [EOS] embedding vector traverses the hidden layer and connects to the output layer. The output from the output layer undergoes a softmax activation to generate the probability vector. The final predicted output corresponds to attack type with the highest probability. For CAN attack detection and classification, the initial step involves tokenizing the CAN messages using the LLAMA 2 tokenizer. The last

token embedding will be extracted by pre-trained LLAMA 2 model. The resulting output embedding of the last token is then forwarded to the classification head. During the training process, the cross-entropy loss is calculated by comparing the predicted labels with the actual ground truth labels.

IV. FINE-TUNING PROCESS

In this section, we explain the fine-tuning process with the aid of mathematical expressions. The main steps comprise the acquisition of class tokens from the pre-trained model, the application of the classification head to acquire the class probability vector, the computation of the loss function, and the weight optimization procedure through the utilization of the AdamW algorithm. Additionally, we introduce methodologies that enhance the efficiency of parameter-based fine-tuning, facilitating the fine-tuning of the LLAMA2 model.

A. Fine-Tuning CAN-C-BERT and CAN-SecureBERT

Initially, the individual raw CAN message is tokenized. This process involves dividing the input sequence into subword tokens using the tokenizer associated with the transformer model. Let X symbolize the tokenized input sequence. Further, let $model$ denote the transformer model, where θ corresponds to the model's parameters. The output, denoted as Z can be formulated as shown in Equation (1).

$$Z = model(X, \theta) \quad (1)$$

Atop the transformer model, a classification head is incorporated to carry out sequence classification. This classification head is composed of a hidden layer, an output layer and subsequently followed by a softmax activation function. Let

W denote the weight matrix of the linear layer, b signify the bias vector, and C represent the total number of classes. The calculated predicted class probabilities P_j for each class j are calculated as expressed in Equation (2).

$$P_j = \frac{e^{(W_j \cdot Z + b_j)}}{\sum_{k=1}^C e^{(W_k \cdot Z + b_k)}} \quad (2)$$

In this study, the chosen loss function is cross-entropy loss, utilized to measure the difference between the predicted probabilities and the actual ground truth labels. Let y_{ij} represent an indicator variable, which takes the value of 1 if the true label for CAN message i corresponds to class j , and 0 otherwise. The cross-entropy loss for an individual CAN message, denoted as L_i , can be defined as depicted in Equation (3).

$$L_i = - \sum_{j=1}^C y_{ij} \log(P_j) \quad (3)$$

The aggregate loss, denoted as L , for a mini-batch of size N is computed as the average of the individual losses L_i . It can be represented as shown in Equation (4).

$$L = \frac{1}{N} \sum_{i=1}^N L_i \quad (4)$$

To mitigate the risk of overfitting, the optimization algorithm employed in this study is AdamW, as presented in [28]. AdamW is an adaptation of the Adam optimizer with the inclusion of weight decay regularization. It facilitates the adjustment of the model's parameters θ to minimize the loss function L .

B. Fine-Tuning CAN-LLAMA2

LLAMA2, with its 7 billion parameters and half-precision, requires 14GB of GPU RAM for its operation. To facilitate training and inference while working with constrained computational resources, the study applies a technique known as LoRA from the parameter-efficient fine-tuning library [28] to fine-tune CAN-LLAMA2.

LoRA involves the approximation of the model's weight matrices through lower-rank matrices. This approximation significantly reduces the number of parameters that need to be retrained from a pre-trained model. Let W denote the weight matrix of a specific layer within the pre-trained model. It can be represented as $W \in R^{m \times n}$, where m signifies the number of output neurons in the layer, and n represents the number of input neurons. The weight matrix W can be approximated by utilizing two lower-rank matrices, U and V , as shown in Equation (5). In this equation, $U \in R^{m \times r}$ and $V \in R^{r \times n}$, with r being the desired rank. The selection of the hyperparameter r is guided by the balance between accuracy and the degree of compression.

$$W = UV \quad (5)$$

The primary goal is to modify the parameters of the low-rank matrices U and V to minimize the cross-entropy loss, as demonstrated in Equation (6), where L denotes the task-specific loss function. In this context, θ_U and θ_V signify the parameters corresponding to U and V , while f represents

the forward pass of the model, incorporating the low-rank approximation.

$$L_{fine-tune} = L(\theta_U, \theta_V) = L(f(UV, X), Y) \quad (6)$$

Throughout the process of updating the model, the parameters θ_U and θ_V are adjusted using the AdamW optimization algorithm. This update is conducted to minimize the fine-tuning loss denoted as $L_{fine-tune}$.

C. Benefits of Fine-Tuning

There are 3 main benefits: transfer learning, better initialization, reduced overfitting. Fine-tuning follows the concept of transfer learning, where knowledge gained while solving one problem is applied to a different task. Comparing with random initialization, the model has learned useful patterns from pre-training dataset and it helps the model converge faster and potentially to a better local minimum. Fine-tuning involves a smaller dataset comparing with the pre-training dataset. This means only small amount model parameters will be changed. The general features learned during pre-training, which can reduce overfitting on the smaller dataset.

In the weight space and the optimization landscape, fine-tuning modifies the pre-trained weights to better suit the new task. It allows the model to find a local minimum that minimize the loss for the specific task. It keeps the majority model parameters and data pattern captured during pre-training. This can be viewed as minimizing the following objective function during fine-tuning:

$$\mathcal{L}_{total} = \lambda \mathcal{L}_{pt} + (1 - \lambda) \mathcal{L}_{ft} \quad (7)$$

where $\mathcal{L}_{pt}$ is the loss from pre-training. $\mathcal{L}_{ft}$ is the loss from fine-tuning. λ is a weighting factor balancing the influence of pre-training and fine-tuning losses.

V. PERFORMANCE METRICS

In this section, we present the metrics selected for performance benchmark. We select metrics that can handle imbalanced dataset and have a direct impact to end users.

The performance evaluation of the proposed CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 models relies on a set of key metrics, including Balanced Accuracy (BA), Precision (PREC), Detection Rate (DR) or Recall, False Alarm Rate (FAR), F1 score, and Model Parameter Size. The mathematical expressions for these metrics can be derived from [30], as represented in Equations (8)- (11), where TP stands for True Positive, TN signifies True Negative, FP represents False Positive, and FN denotes False Negative.

Given the inherent characteristics of the hacking dataset, it exhibits a significant imbalance, featuring notably fewer instances of attack data in comparison to normal data. As a result, the conventional accuracy metric can be misleading in such a scenario. To address this, BA is employed, which represents the mean accuracy of individual class predictions. BA offers a more reliable measure of model performance, particularly suited for datasets characterized by an imbalanced class distribution.

$$BA = \frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FN_i} \quad (8)$$

The *PREC* measures the accuracy of positive predictions generated by a model.

$$PREC = \frac{TP}{TP + FP} \quad (9)$$

The *DR*, commonly known as Recall, assesses the proportion of *TP* predictions relative to the entirety of actual positive cases.

$$DR = \frac{TP}{TP + FN} \quad (10)$$

The *FAR* quantifies the proportion of *FP* predictions relative to the entire set of actual negative cases. This metric holds significant importance, particularly in the context of an IDS. For instance, a model with a *FAR* of $1e-3$ may be deemed exceptional performance in the majority of ML tasks. However, in the case of IDS, the practical implications become apparent. Consider the scenario where millions of CAN messages are generated daily by a multitude of vehicles. To illustrate, let us assume a daily volume of 10 million messages. With a *FAR* of $1e-3$, the number of false alarm messages would amount to 10,000. Managing and triaging such a volume on a daily basis becomes operationally impractical for a VSOC team.

$$FAR = \frac{FP}{FP + TN} \quad (11)$$

The F1 score represents the harmonic mean of *PREC* and *DR* or Recall. This metric furnishes a balanced and comprehensive measure of a model's performance.

$$F1 = \frac{2 \cdot PREC \cdot DR}{PREC + DR} \quad (12)$$

Imbalanced dataset is used to evaluate the model's performance in real world scenario. Per attack type performance in terms of BA, DR, FAR, F1 is reported to show and compare model performance for major and minor attack types.

VI. DATASETS

In this study, we employed 2 datasets to perform experiments to evaluate the performance of proposed models. The first is the Car Hacking Dataset, which was gathered from Hyundai's YF Sonata and made available by Song et al. [13]. This dataset encompasses three distinct categories of attacks, namely, DoS attacks, Fuzzy attacks, and Spoofing attacks. These datasets were curated by capturing CAN traffic through the On-Board Diagnostics II (OBD-II) port of a real vehicle while simulated message attacks were actively being executed. The data attributes within the dataset encompass the following fields: Timestamp, CAN ID, Data Length Code (DLC), DATA[0], DATA [1], DATA [2], DATA [3], DATA [4], DATA [5], DATA [6], DATA [7], and Flag. Further insights into each attack type, as detailed in [13], are outlined as follows:

- 1) *DoS Attack*: The DoS attack involves the injection of high-priority CAN messages, exemplified by the '0x000' CAN ID packet, at brief intervals. Specifically, these '0x000' CAN ID messages were injected every 0.3 milliseconds.
- 2) *Fuzzy Attack*: The Fuzzy attack entails the injection of messages featuring spoofed, randomly generated CAN

TABLE II
CAR HACKING DATASET

Attack Type	Messages	Normal	Injected
DoS Attack	3665771	3078250	587521
Fuzzy Attack	3838860	3347013	491847
Spoofing Drive Gear	4443142	3845890	597252
Spoofing RPM Gauge	4621702	3966805	654897
Attack-Free (Normal)	988987	988872	NA

TABLE III
CAN-TRAIN-AND-TEST DATASET

Attack Type	Messages
Benign	49042866
DoS Attack	198669
Fuzzy Attack	123647
Spoofing RPM Gauge	13680
Spoofing Drive Gear	2561

ID and DATA values. This injection was carried out at intervals of 0.5 milliseconds, including messages with randomized CAN ID and data values.

- 3) *RPM/Gear Attack*: The RPM/Gear attack revolves around the injection of messages associated with specific CAN IDs pertinent to RPM and Gear information. Messages related to RPM and Gear were introduced at intervals of 1 millisecond.

These attacks were executed by introducing messages into the CAN network, aiming to replicate real-world scenarios and behaviors within the Hyundai YF Sonata. An overview of the Car Hacking dataset is presented in Table II.

The second dataset is the can-train-and-test dataset [31]. It contains controller area network (CAN) traffic collected from 2 OEMs and 4 different vehicle types including the 2017 Subaru Forester, the 2016 Chevrolet Silverado, the 2011 Chevrolet Traverse, and the 2011 Chevrolet Impala. The format of dataset fields and labels are converted to be consistent with Car Hacking Dataset. For attack types, we keep only BENIGN, DoS, Fuzzy, RPM and Gear to be able to compare it with CAR Hacking Dataset. An overview of can-train-and-test dataset is presented in Table III.

VII. EXPERIMENTS AND RESULTS

In this section, we present how to process the dataset to train and evaluate our proposed models. We show the experimental setup, hyperparameters employed, as well as model complexity. Subsequently, we delve into a comprehensive discussion of the experimental results and emphasize noteworthy observations.

A. Fine-Tuning Hyperparameters

We partitioned the Car Hacking Dataset into a 70% training dataset and a 30% test dataset. In practical applications, collecting data from vehicles is often a challenging endeavor. It is crucial to explore how the quantity of training data affects model performance. Therefore, our proposed models were trained and validated using subsets amounting to 1% and 10% of the training dataset, which were randomly selected from

TABLE IV
MODEL SIZE

Model Type	Model Parameters	Trainable Parameters	Training Time	Inference Speed
CAN-C-BERT	110 Million	110 Million	4 mins	894 messages/s
CAN-SecureBERT	123 Million	123 Million	5 mins	965 messages/s
CAN-LLAMA2	7 Billion	40 Million	118 mins	14 messages/s

the entire dataset. The random selection process maintained a balanced representation of data, with the normal data comprising a ratio ten times smaller than other attack-type data. For instance, when the selected attack types amounted to 1%, the normal dataset selected was 0.1%, ensuring a balanced distribution of normal and attack-type data. Subsequently, the selected dataset was further divided into a 70% training portion and a 30% validation portion. The entire 30% test dataset from the Car Hacking dataset was reserved to evaluate the trained and validated models.

The hardware used for training in this study possesses the following specifications: AMD Ryzen 9 5900X 12-Core Processor with a clock speed of 3.70 GHz, 128GB of RAM, and two Nvidia RTX 3090 GPUs, each equipped with 24GB of VRAM and interconnected using the Nvidia SLI bridge.

For the training hyperparameters, we define a search space as following for each parameter. `lora_r` is from 1 to 64. `lora_alpha` is 1 to 128, `lora_dropout` is 0.0 to 0.5. `gradient_accumulation_steps` is 1 to 8. `learning_rate` is $1e-6$ to $1e-4$. `weight_decay` is $1e-6$ to $1e-2$. `per_device_train_batch_size` is 4, 8, 16. `per_device_eval_batch_size` is 16, 32, 64. The hyperparameter space is then sampled with Tree-structured Parzen Estimator (TPE) sampler which is a type of bayesian optimization with objective to minimize train and validation loss. For the batch size, we also consider the max size that can be handled by our hardware efficiently. An example of hyperparameter importance for CAN-LLAMA2 is shown in Figure 7.

The identical settings are applied to both CAN-C-BERT and CAN-SecureBERT. These models utilize a training batch size of 4 and a validation batch size of 32. The learning rate is set at $5e-5$, while the weight decay is configured at 0.01. As for CAN-LLAMA2, the training batch size is set to 4, the validation batch size is 16, and a gradient accumulation step of 4 is implemented. The learning rate for CAN-LLAMA2 is established at $3e-5$, and the weight decay is fixed at 0.01. Moreover, to facilitate the training of CAN-LLAMA2 while accommodating limited computational resources, the model parameters are loaded in 4-bit precision. LoRA, as discussed in Section IV, is incorporated into the training pipeline. Specifically, the LoRA attention dimension is set to 16, the alpha parameter for LoRA scaling is established at 64, a dropout probability of 0.1 is applied to LoRA layers, and the bias is maintained at a value of 0. All three models are trained for a total of 10 epochs.

B. Model Complexity

Table IV presents a comparative analysis of fine-tuned model sizes and parameters. Both CAN-C-BERT and CAN-SecureBERT offer the ability to fine-tune all their parameters

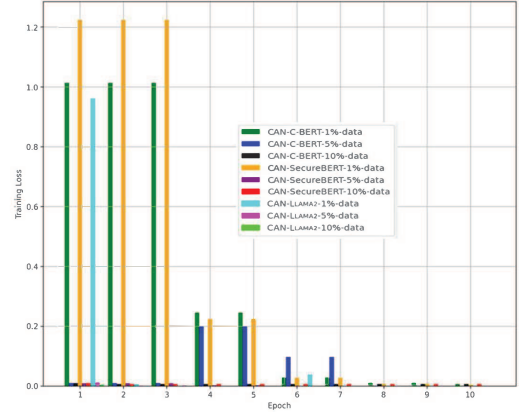


Fig. 8. Train Loss Comparison between CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 with 1%, 5% and 10% Training Data.

during training. Notably, when employing a 1 percent training dataset, the training times for these models are approximately 4 and 5 minutes, respectively. In contrast, training the CAN-LLAMA2 model requires considerably more time, with a training duration of 118 minutes. However, it is important to note that the inference speed of CAN-LLAMA2 is approximately eight times slower than the other two models. This difference is primarily attributed to computational resource limitations, given that CAN-LLAMA2 concludes 7 billion parameters. After implementing LoRA, it becomes possible to fine-tune approximately 40 million parameters from the linear layers of the model. Nevertheless, it is worth emphasizing that the computational demands of CAN-LLAMA2 in terms of matrix multiplications, activations, and other mathematical operations are significantly higher than those of the other two models. Furthermore, due to restrictions related to GPU memory size, CAN-LLAMA2 can only accommodate a maximum batch size of 16 CAN messages for validation. This limitation not only affects memory access times but also gives rise to memory bottlenecks.

Another noteworthy observation is that only 0.57% of the parameters are altered in the CAN-LLAMA2 model during fine-tuning. This implies that the majority of the original LLAMA2 model's parameters remain unchanged. Consequently, CAN-LLAMA2 can be repurposed for other language-related tasks. The VSOC team can utilize the same model and undertake various downstream tasks by fine-tuning pre-trained models and incorporating adapter heads.

C. Results

In this section, we conduct a thorough analysis of our results to address our research questions and gain insights into the performance of our proposed models against the

TABLE V
MODEL PERFORMANCE COMPARISON

Model Type	BA	PREC	DR	FAR	F1
DCNN[13]	0.999300	-	0.998400	1.60e-3	0.999100
E-MLP[14]	0.990000	0.990000	0.990000	3.00e-3	0.990000
GIDS[15]	0.975250	0.976250	0.986500	1.00e-3	0.987900
Transformer Sequential CAN ID[16]	-	0.998980	999350	5.80e-4	0.999170
KNN[17]	0.974000	-	0.963000	5.30e-2	0.934000
SVM[17]	0.965000	-	0.957000	4.80e-2	0.933000
XYF-K[18]	0.991000	-	0.983900	-	0.987900
SAIDuCANT[19]	0.872100	-	0.866600	1.70e-2	0.920000
SSAE[20]	-	-	0.985000	1.76e-2	0.920000
LSTM-Autoencoder[21]	0.990000	-	0.990000	-	0.990000
MTH-IDS[12]	0.999990	-	0.999990	6.00e-4	0.999990
CAN-C-BERT-1%-data	0.991377	0.992874	0.991377	9.90e-3	0.991773
CAN-SecureBERT-1%-data	0.999718	0.999718	0.999718	1.30e-4	0.999718
CAN-LLAMA2-1%-data	0.999587	0.999588	0.999587	3.10e-4	0.999587
CAN-C-BERT-5%-data	0.999865	0.999911	0.999911	2.35e-4	0.999911
CAN-SecureBERT-5%-data	0.999973	0.999982	0.999982	3.20e-5	0.999982
CAN-LLAMA2-5%-data	0.999989	0.999989	0.999989	2.55e-5	0.999989
CAN-C-BERT-10%-data	0.999965	0.999965	0.999965	3.60e-5	0.999965
CAN-SecureBERT-10%-data	0.999991	0.999991	0.999991	3.50e-6	0.999991
CAN-LLAMA2-10%-data	0.999993	0.999993	0.999993	3.10e-6	0.999993

TABLE VI
ATTACK CLASSIFICATION PERFORMANCE - CAR HACKING DATASET - CAN-C-BERT

Attack Type	Validation Instances	PREC	DR	FAR	F1
DoS	117674	0.999230	1.0	3.1e-5	0.999614
Fuzzy	98867	0.999868	0.999888	4.5e-6	0.999878
Gear Spoofing	119707	1.0	1.0	0	1.0
RPM Spoofing	131324	1.0	1.0	0	1.0

TABLE VII
ATTACK CLASSIFICATION PERFORMANCE - CAR HACKING DATASET - CAN-SECUREBERT

Attack Type	Validation Instances	PREC	DR	FAR	F1
DoS	117674	1.0	1.0	0	1.0
Fuzzy	98867	0.999898	0.999828	3.5e-6	0.999878
Gear Spoofing	119707	1.0	1.0	0	1.0
RPM Spoofing	131324	1.0	1.0	0	1.0

TABLE VIII
ATTACK CLASSIFICATION PERFORMANCE - CAR HACKING DATASET - CAN-LLAMA2

Attack Type	Validation Instances	PREC	DR	FAR	F1
DoS	117674	1.0	1.0	0	1.0
Fuzzy	98867	0.999909	0.999858	3.1e-6	0.999883
Gear Spoofing	119707	1.0	1.0	0	1.0
RPM Spoofing	131324	1.0	1.0	0	1.0

TABLE IX
ATTACK CLASSIFICATION PERFORMANCE - CAN-TRAIN-AND-TEST DATASET - CAN-C-BERT

Attack Type	Validation Instances	PREC	DR	FAR	F1
DoS	576	0.991394	1.0	3.3e-4	0.995678
Fuzzy	362	0.972144	0.964088	6.5e-4	0.968099
Gear Spoofing	11	1.0	0.818181	0	0.988764
RPM Spoofing	44	0.977777	1.0	6.4e-5	0.988764

2 datasets selected. The validation results for Car Hacking Dataset, including training loss, validation loss, BA, PREC, DR, and F1 score, are visualized in Figures 8–13. The detailed performance per attack type classification is shown in Tables VI, VII, VIII. To evaluate the generalization capability of the proposed models against various vehicle types and OEMs, the

detail performance per attack type classification is shown in Table IX, X and XI.

1) *Test Directly Without Fine-Tuning*: To understand the benefits of fine-tuning using our proposed models with added classification heads, we perform tests to directly use pre-trained models with added classification heads but without

TABLE X
ATTACK CLASSIFICATION PERFORMANCE - CAN-TRAIN-AND-TEST DATASET - CAN-SECUREBERT

Attack Type	Validation Instances	PREC	DR	FAR	F1
DoS	576	0.996539	1.0	1.3e-4	0.998266
Fuzzy	362	0.991643	0.983425	1.9e-4	0.987517
Gear Spoofing	11	1.0	0.909090	0	0.952380
RPM Spoofing	44	0.977777	1.0	6.4e-5	0.988764

TABLE XI
ATTACK CLASSIFICATION PERFORMANCE - CAN-TRAIN-AND-TEST DATASET - CAN-LLAMA2

Attack Type	Validation Instances	PREC	DR	FAR	F1
DoS	576	0.998266	1.0	6.6e-5	0.999132
Fuzzy	362	0.994429	0.986187	1.3e-4	0.990291
Gear Spoofing	11	1.0	0.909090	0	0.952380
RPM Spoofing	44	0.977777	1.0	6.4e-5	0.988764

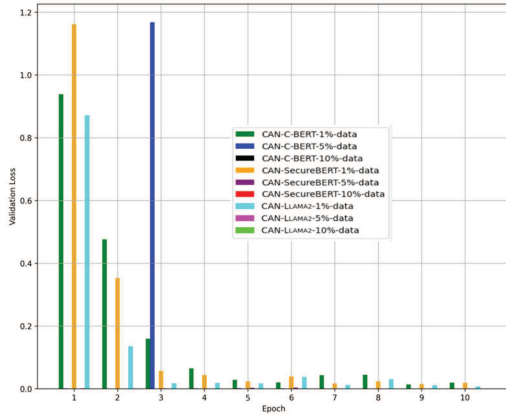


Fig. 9. Validation Loss Comparison between CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 with 1%, 5% and 10% Training Data.

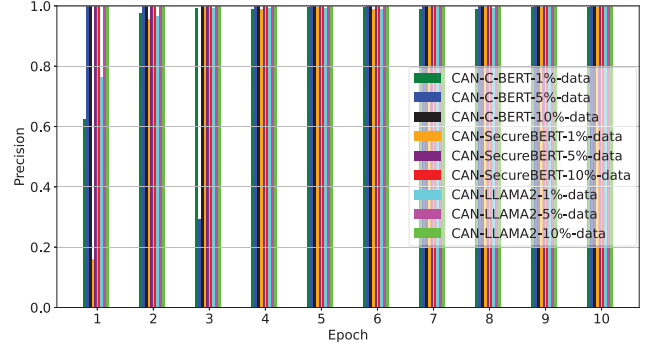


Fig. 11. Precision Comparison between CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 with 1%, 5% and 10% Training Data.

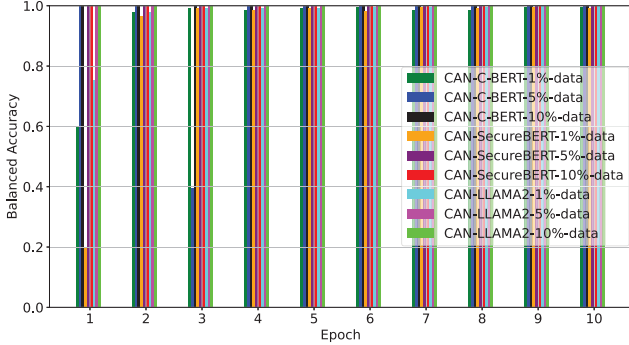


Fig. 10. Balanced Accuracy Comparison between CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 with 1%, 5% and 10% Training Data.

fine-tuning. We then use these models to predict using test datasets. We observe that all models made random predictions for all CAN message instances. This finding indicates that large pre-trained models can not be directly used to classify vehicle CAN attacks. Fine-tuning has to be performed for the models to learn the attack data patterns.

2) *Training Loss and Validation Loss*: Training loss and validation loss result are shown in Figure 8 and 9. All models have training and validation loss close to 0 within 10 epochs,

which indicates that CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 all converge within ten epochs. Figure 8 clearly illustrates that the models trained with 10% of the data converge more swiftly than those trained with only 1% of the data. All six training losses are close to 0, indicating that they have all converged. However, for validation loss from Figure 9, the models trained with 10% data have loss close to 0, which are much less than the models trained with 1% data. This indicates that models trained with 10% exhibit more accurately. The validation loss is close to 0. It means these models can perform effectively on the unseen data within the validation dataset.

3) *Balanced Accuracy, Precision, Detection Rate and F1 Score*: The BA, PREC, DR, and F1 score are plotted in Figure 10 to 13. All 6 models have achieved close to the value 1 result for these metrics. It indicates all models are very accurate. From the plots of model performance using 1% training data in Figures 10-13, we can observe CAN-LLAMA2 exhibits superior early-stage performance and converges to the value of 1.0 more rapidly compared to the other two models. The models trained with 10% data perform better than the models trained with 1% and 5% data for all metrics. Among all six models, CAN-LLAMA2 trained with 10% data performs the best. CAN-SecureBERT trained with 10% data is the second best among all metrics.

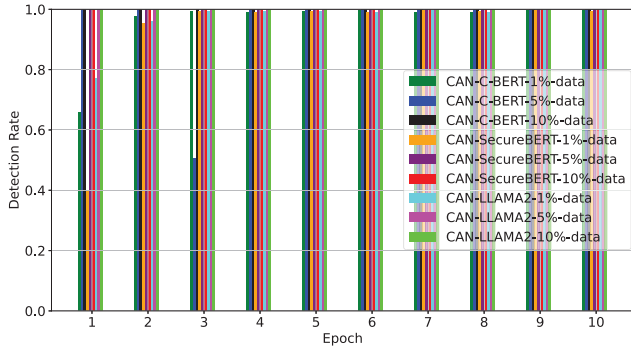


Fig. 12. Detection Rate Comparison between CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 with 1%, 5% and 10% Training Data.

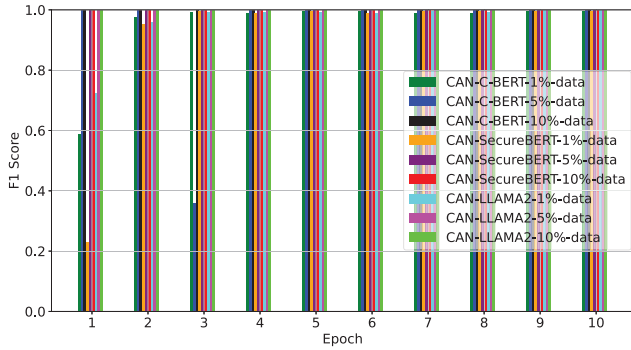


Fig. 13. F1 Score Comparison between CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 with 1%, 5% and 10% Training Data.

Table V provides a comprehensive summary of the models' performance after testing, with a comparison to state-of-the-art models. CAN-LLAMA2, trained with 10% of the data, outperforms all other state-of-the-art models, achieving BA, PREC, DR, and F1 scores of 0.999993, with a FAR of $3.1e-6$. This remarkable performance implies that, with a dataset of 10 million CAN messages, only 31 messages are expected to be false alarms. The second-best model is CAN-SecureBERT, also trained with 10% of the data. In this case, there are 35 expected false alarms for 10 million CAN messages. Both models significantly outperform MTH-IDS and CAN-C-BERT, thereby reducing the workload for VSOC team during the triage process. These findings underscore the effectiveness of a more complex model with pre-trained knowledge for vehicle network intrusion detection, surpassing an IDS designed with a combination of multiple models and algorithms. It demonstrates that a single base model, with fine-tuned parameter sets or adapters, can handle multiple downstream tasks.

The detailed model performance for the classification of each attack type is listed in Tables VI, VII, and VIII. CAN-SecureBERT and CAN-LLAMA2 both achieve 100% performance for DoS, Gear Spoofing, and RPM spoofing attacks. However, both models make some few incorrect predictions for Fuzzy attacks, which involve the injection of randomly spoofed messages.

4) *Result When Testing With Various Vehicle Types:* We follow the same process to fine-tune our proposed models

Normal	timestamp: 1672531212.2400131,CAN ID: 0F1, DLC: 4,data1: 3E,data2: 39,data3: 00,data4: C0
DoS	timestamp: 1672531333.3916512,CAN ID: 0AA, DLC: 8,data1: 2F,data2: 04,data3: 2D,data4: 90, data5: 02,data6: 52,data7: 6F,data8: 00
Fuzzy	timestamp: 1672531293.182986,CAN ID: 799, DLC: 2,data1: 81,data2: 7B
Spoof Gear	timestamp: 1672531208.712928,CAN ID: 0D1, DLC: 4,data1: C0,data2: 01,data3: 00,data4: 00
Spoof RPM	timestamp: 1672531381.805054,CAN ID: 0C9, DLC: 7,data1: 80,data2: 00,data3: 00, data4: 00,data5: 00,data6: 00,data7: 18

Fig. 14. Shap values of input can message token for each of message type.

using can-train-and-test dataset [29]. Here, we present the results when training using 1% of the dataset and test it on the validation dataset. We found our proposed models have good generalization capability against the CAN messages generated from mixed OEMs and vehicle types. As shown in Tables IX, X, and XI, all models achieve 100% performance for DoS, Gear Spoofing, and RPM spoofing attacks. However, all models make some few incorrect predictions for Fuzzy attacks. CAN-LLAMA2 performance is the best overall. This finding is consistent with what we find when test on Car Hacking Dataset.

D. Model Prediction Explanation

In this study, SHAP values (SHapley Additive Explanations) [32] are used to explain the output of the proposed models by attributing the prediction to each input CAN message. SHAP values, derived from cooperative game theory, provide a fair distribution of each feature's contribution to the overall prediction, making the model more interpretable. This helps us understand how specific fields in the CAN message influence the classification of normal versus attack messages. Fig. 14 shows the SHAP values of example CAN messages. The blue highlights indicate that the tokens contribute positively to the prediction result, while the red highlights indicate a negative contribution. The darker the color, the greater the impact of the token on the prediction.

For a normal message, the model primarily relies on the pattern of DLC and data1 to data4. In this context, the value of data3 being "00" has the largest negative effect on the model's classification of the message as normal, while data1 has the largest positive effect. These data fields often represent regular operational parameters in a vehicle, such as sensor readings or control signals. For a DoS attack, the model analyzes the CAN ID, DLC, and data1 to data8. DoS attacks typically aim to overwhelm the bus with a high volume of messages, so the CAN ID often has the largest positive effect, while data8 exerts a large negative effect, potentially indicating the presence of an anomaly or malformed data in the message. Fuzzy attacks, which introduce irregularities in data values to confuse the

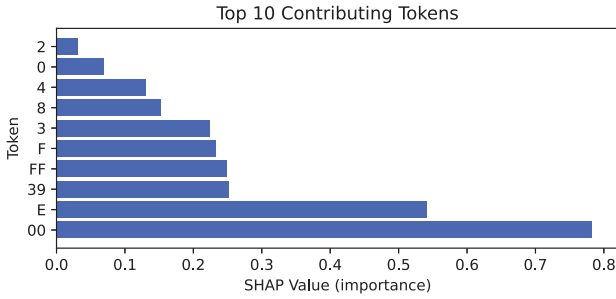


Fig. 15. Top 10 contributing tokens among all can messages.

system, are mainly detected through the timestamp, CAN ID, DLC, and data1 to data2. The timestamp and data1 have the largest impact on the prediction, as these fields are crucial for detecting slight anomalies in message timing or values that could indicate fuzzing attempts. For spoof gear messages, the prediction relies on data1 and data4, where data1 has the largest positive effect and data4 the largest negative effect. These data fields are likely involved in controlling or reporting the vehicle's gear position, which is commonly spoofed in attacks targeting system integrity. Spoof RPM messages are influenced by data2, data3, and data7, where data7 has the largest positive effect, while data2 and data3 have the largest negative effects. These fields are related to vehicle speed or RPM readings, which are often spoofed in attacks to mislead the vehicle's control systems. Fig. 15 shows the top 10 tokens with the highest SHAP values across all message types are: 00, E, 39, FF, F, 3, 8, 4, 0, and 2. These hexadecimal values represent common patterns or anomalies in the data fields that correspond to specific attack signatures.

E. Discussions

1) *Model Performance Comparison:* Based on the above results, there is a significant difference about the changing trends of the training loss and the validation loss from Figure 8 and 9. This difference indicates that the models trained with a larger dataset tend to generalize more effectively than those trained with a smaller dataset. This conclusion is substantiated by the analysis of other model performance metrics, where the models trained with 10% of the data consistently outperform their 5% and 1% data counterparts. The more training data used will contribute to model performance improvement but the improvement magnitude gradually reduce as training data increases.

All proposed models achieve exceptional performance in CAN message log classification. These models operate directly on raw, text-based CAN messages, and their performance metrics all exceed 0.99, as shown in Figures 10- 13. This feature indicates that transformer-based models can directly classify CAN message logs without the need for feature engineering and data preprocessing.

When looking at the model performance with 1% data from Figure 10 to 13, CAN-LLAMA2 converges faster than the rest of models with 1% data. This indicates that larger models, such as CAN-LLAMA2, have a better capability to capture complex

CAN message patterns. The increased number of model layers in CAN-LLAMA2 facilitates more efficient information sharing across layers due to its larger parameter size, contributing to its faster convergence.

The model with more pre-trained knowledge from transformer models perform better compared to models with less pre-trained knowledge. LLAMA2 is pre-trained on a larger dataset compared to BERT and SecureBERT. To capture more pre-trained knowledge, LLAMA2 has 54 times more parameters than BERT and SecureBERT. However, only 40 million parameters are allowed to be fine-tuned after applying LoRa, with the remaining parameters being frozen. Despite the reduced number of fine-tuned parameters, CAN-LLAMA2 outperforms CAN-C-BERT and CAN-SecureBERT. This observation indicates that a larger transformer-based model, trained on an extensive dataset, can enhance CAN classification performance.

CAN-LLAMA2 can capture more DoS and Fuzzy attack messages than other proposed models. LLAMA 2's significantly larger parameter count (e.g., 7B) enables it to learn more expressive and nuanced feature representations from the tokenized CAN messages. This is particularly beneficial for detecting subtle irregularities introduced by attacks like Fuzzy injection or the atypical message frequency seen in DoS attacks. LLAMA 2 pretraining covers a wide range of structured, semi-structured, and code-like data. This diverse exposure enhances its ability to generalize to domain-specific message formats, such as those found in CAN traffic, even when fine-tuned on relatively small labeled datasets. Unlike encoder-only models (e.g., SecureBERT), LLAMA 2 uses a decoder-based architecture that generates outputs token-by-token. Even though classification is performed per message, this architecture tends to be more sensitive to fine-grained variations in token sequences, making it effective for identifying malformed or anomalous payloads characteristic of DoS and Fuzzy attacks. The self-attention mechanisms in LLAMA 2 can capture complex relationships between tokens within a single message. This enhances its capability to detect internal inconsistencies or deviations that often go unnoticed in smaller models like SecureBERT.

CAN-SecureBERT uses SecureBERT which is primarily trained with cybersecurity-focused data. While it exhibits slightly worse performance than CAN-LLAMA2, it outperforms CAN-C-BERT. However, SecureBERT has a parameter size that is 20% larger than CAN-C-BERT. Consequently, it remains uncertain whether the performance difference can be attributed to the pre-trained domain knowledge or the increased number of parameters in the model.

2) *Model Scalability and Unknown Attack Detection:* All proposed models have millions of parameters, as shown in Table IV. To achieve higher inference speeds, a dedicated graphics processing unit (GPU) is required on the device or vehicle. These constraints limit the feasibility of deploying the proposed models in vehicles. To overcome these challenges, a hybrid intrusion detection system-based architecture [14] can be utilized. This architecture consists of two layers: an unknown attack or anomaly detection layer deployed on the vehicle and a classification layer hosted on an edge or

cloud server. The proposed model can be implemented at the edge or cloud server to classify abnormal CAN messages. The edge and cloud servers can be horizontally scaled, ensuring optimal scalability and accommodating varying computational demands. This design allows for the deployment of a lightweight, vehicle-based model to detect abnormal CAN messages. The study in [14] proposed an autoencoder-based model to detect unknown attack types as abnormal messages. The proposed models deployed on edge and cloud servers focus on classifying abnormal messages received from vehicles. Their performance can be monitored and retrained continuously to maintain high accuracy.

3) *Data Privacy and Security*: The proposed models use CAN messages as input without the need to access vehicle or user related information. The data transmission between vehicle and OEM cloud server need to be protected. We propose the following techniques for this purpose. First, all CAN data can be anonymized to prevent leakage of sensitive or personally identifiable information. Common techniques such as field masking or data tokenization can be applied to use name, address, VINs or geolocation. Secondly, the data transmissions can be secured using industry-standard protocols such as TLS (Transport Layer Security) with data in transit encrypted, Mutual TLS (mTLS) with requirement to authenticate both client and server endpoints. Lastly, it's critical to ensure the data transmission architecture meets UNECE WP.29 for vehicle cybersecurity. This regulation enforces secure communication channels, risk management for external interfaces and threat detection and response includes logging, monitoring, and alerting for suspicious data behavior.

VIII. CONCLUSION

In this study, we propose a novel approach for CAN intrusion detection and attack classification by fine-tuning pre-trained transformer-based models. Three distinct models, namely CAN-C-BERT, CAN-SecureBERT, and CAN-LLAMA2 have been developed. Our proposed models can directly use CAN message logs and eliminate the need to perform data preprocessing. After trained by using pre-balanced CAN dataset, their performances have been compared against state-of-the-art models. CAN-LLAMA2 exhibits the highest level of performance, surpassing all empirical state-of-the-art IDS systems. CAN-SecureBERT stands as the second-best model for performance and the best model for inference speed. The leading model, CAN-LLAMA2, achieves outstanding results with a BA, PREC, DR, and F1 score of 0.999993, accompanied by an impressively low FAR of $3.1e-6$. This FAR is approximately 52 times better than that of MTH-IDS, clearly outperforming all other state-of-the-art models. Overall, our study advances the field of CAN IDS, offering insights into model design, performance, and adaptability for cybersecurity applications.

FUTURE WORKS

One of the key limitations of our research is the computational resource constraints. In our forthcoming research, we will explore methods to further reduce the model size

and enhance inference speed of the proposed CAN-LLAMA2 model. Furthermore, we plan to enhance the system's robustness by evaluating the models under various adversarial attack scenarios. Specifically, we aim to test the models' resilience to both input-level perturbation attacks and protocol-level adversarial manipulations that mimic realistic CAN attack strategies on in-vehicle networks. The results of these evaluations will inform the design of more secure and resilient CAN intrusion detection models.

ACKNOWLEDGMENT

Any opinions, findings, and conclusions or recommendations expressed in this work are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

- [1] *UN Regulation, no. 155—CyberSecurity and CyberSecurity Management System*. Accessed: Oct. 30, 2023. [Online]. Available: <https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security>
- [2] W. Wu et al., "A survey of intrusion detection for in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 919–933, Mar. 2020.
- [3] K.-T. Cho and K. G. Shin, "Viden: Attacker identification on in-vehicle networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1109–1123.
- [4] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2016, pp. 63–68.
- [5] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.
- [6] N. Shrirish Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," 2019, *arXiv:1909.05858*.
- [7] S. Niu, Y. Liu, J. Wang, and H. Song, "A decade survey of transfer learning (2010–2020)," *IEEE Trans. Artif. Intell.*, vol. 1, no. 2, pp. 151–166, Oct. 2020.
- [8] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [10] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, "SecureBERT: A domain-specific language model for cybersecurity," in *Proc. Int. Conf. Secur. Privacy Commun. Syst.* Cham, Switzerland: Springer, 2022, pp. 39–56.
- [11] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [12] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
- [13] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198.
- [14] X. Li and H. Fu, "A hybrid ensemble multilayer-perceptron-based intrusion detection system for vehicle networks," in *Proc. Int. Inst. Cognit. Informat. Cognit. Comput.*, Apr. 2023, pp. 1–16.
- [15] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. 16th Annu. Conf. Privacy, Secur. Trust (PST)*, Aug. 2018, pp. 1–6.
- [16] T. P. Nguyen, H. Nam, and D. Kim, "Transformer-based attention network for in-vehicle intrusion detection," *IEEE Access*, vol. 11, pp. 55389–55403, 2023.
- [17] A. Alshammari, M. Zohdy, D. Debnath, and G. Corser, "Classification approach for intrusion detection in vehicle systems," *Wireless Eng. Technol.*, vol. 9, no. 4, pp. 79–94, 2018.
- [18] V. S. Barletta, D. Caivano, A. Nannavecchia, and M. Scalera, "A Kohonen SOM architecture for intrusion detection on in-vehicle communication networks," *Appl. Sci.*, vol. 10, no. 15, p. 5062, Jul. 2020.

- [19] H. Olufowobi, C. Young, J. Zambreno, and G. Bloom, "SAIDuCANT: Specification-based automotive intrusion detection using controller area network (CAN) timing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1484–1494, Feb. 2020.
- [20] S. F. Lokman, A. T. Othman, M. H. Abu Bakar, and R. Razuwan, "Stacked sparse autoencodersbased outlier discovery for in-vehicle controller area network (CAN)," *Int. J. Eng. Technol.*, vol. 7, no. 4, pp. 375–380, 2018.
- [21] J. Ashraf, A. D. Bakhshi, N. Moustafa, H. Khurshid, A. Javed, and A. Beheshti, "Novel deep learning-enabled LSTM autoencoder architecture for discovering anomalous events from intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4507–4518, Jul. 2021.
- [22] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
- [23] E. Nwafor and H. Olufowobi, "CANBERT: A language-based intrusion detection model for in-vehicle networks," in *Proc. 21st IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2022, pp. 294–299.
- [24] N. Alkhatib, M. Mushtaq, H. Ghauch, and J.-L. Danger, "CAN-BERT do it? Controller area network intrusion detection system based on BERT language model," in *Proc. IEEE/ACS 19th Int. Conf. Comput. Syst. Appl. (AICCSA)*, Dec. 2022, pp. 1–8.
- [25] S. Khandelwal and S. Shreejith, "A lightweight FPGA-based IDS-ECU architecture for automotive CAN," in *Proc. Int. Conf. Field-Programmable Technol. (ICFPT)*, Dec. 2022, pp. 1–9, doi: [10.1109/ICFPT56656.2022.9974508](https://doi.org/10.1109/ICFPT56656.2022.9974508).
- [26] S. Huan et al., "T-shaped CAN feature integration with lightweight deep learning model for in-vehicle network intrusion detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 12, pp. 21183–21196, Dec. 2024, doi: [10.1109/TITS.2024.3478371](https://doi.org/10.1109/TITS.2024.3478371).
- [27] Y. Zhu et al., "Aligning books and movies: Towards story-like visual explanations by watching movies and reading books," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 19–27.
- [28] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.
- [29] S. Mangrulkar. (2022). *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning Methods*. [Online]. Available: <https://github.com/huggingface/peft>
- [30] F. Salo, M. Injadat, A. Bou Nassif, and A. Essex, "Data mining with big data in intrusion detection systems: A systematic literature review," 2020, *arXiv:2005.12267*.
- [31] B. Lampe and W. Meng, "Can-train-and-test: A curated CAN dataset for automotive intrusion detection," *Comput. Secur.*, vol. 140, May 2024, Art. no. 103777.
- [32] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–10.



Xuemei Li (Student Member, IEEE) received the Master of Science degree in computer science from Oakland University, USA, in 2021. She is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering, Oakland University. Her research interests lie at the intersection of artificial intelligence and cybersecurity, with a particular emphasis on connected vehicles, financial fraud detection, machine learning, and large language models. She is especially focused on developing interpretable and privacy-aware AI systems. Her recent work explores prompt engineering, fine-tuning of LLMs, and reinforcement learning to enhance real-time decision-making in high-stakes domains, such as finance and intelligent transportation.



Huirong Fu (Senior Member, IEEE) received the Ph.D. degree from Nanyang Technological University, Singapore, in 2000. She is a Distinguished Professor with the Department of Computer Science and Engineering, Oakland University (OU), Rochester, MI, USA, where she has led research in cybersecurity, trust management, and privacy since joining as an Assistant Professor in 2005. Previously, she was an Assistant Professor at North Dakota State University for three years and a Post-Doctoral Research Associate at Rice University for two years.

She was as the Principal Investigator on more than 20 federally funded projects. She is the Founding Director of the OU Center for Cybersecurity and led the university's designation as a National Center of Academic Excellence in Cyber Defense (NCAE-C). She has authored more than 100 peer-reviewed publications.